

AI + Backend Engineer

Six-Month Build-First Course

A 100-Days-of-Code-style curriculum for backend + AI engineering

The single rule

Learn a concept because the current project needs it. Build it, measure it, break it, explain it, then reuse it in the next project.

Six-month outcome

Your GitHub should show a coherent progression: backend → ML/LLM foundations → evaluated RAG → agents → research → fine-tuning/inference → protocols → production AI.

Baseline

10–14 focused hours/week. Six build days + one lighter review/rest day. If a project needs extra time, finish it rather than rushing to the next technology.

How to use this document

- Follow the weeks in order. The architecture is intentionally cumulative.
- Every study day produces an artifact: code, test, experiment, metric, diagram, decision or written explanation.
- Use AI coding tools, but define the task and acceptance criteria yourself. Review generated code and run tests.
- Keep a decision log: what you tried, what failed, what changed, and why.
- Do not move on because a demo works. Use the completion gate.
- Keep frontend work minimal unless it helps demonstrate the backend/AI system.

The connection across six months

Month 1 builds the backend/document-processing substrate. **Month 2 builds the ML**, tokenization and Transformer mental model. **Month 3** reuses Month 1 directly and uses Month 2 to **understand what the LLM is doing**. The Berkshire project then becomes the knowledge layer for the support agent. The support agent teaches state, tools, permissions and orchestration; those become the base for the research agent and Ask-the-Web. Deep Research produces failure/evaluation data that motivates fine-tuning. The fine-tuned model is then served through the same model interface. MCP and A2A expose existing capabilities rather than creating toy projects. Month 6 assembles the components into one production-oriented system.

Dependency map

Document Processing Backend



ML + LLM Foundations



Berkshire Research Assistant v2

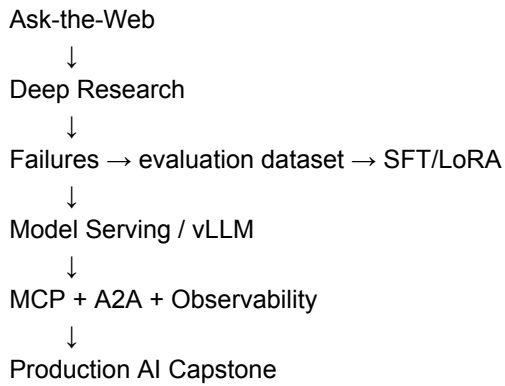


Customer Support Agent



Tool-Using Research Agent





What is reused

Stage	Reusable asset
Month 1	FastAPI structure, auth, PostgreSQL, migrations, file storage, background jobs, tests, Docker, logging.
Month 2	Tokenizer/model vocabulary, PyTorch training/evaluation habits, Mini-GPT mental model.
Berkshire	Document ingestion, chunks, metadata, retriever interface, evidence/citation objects, LLM adapter, evaluation set.
Support Agent	Berkshire retriever as knowledge tool; same API/DB/auth patterns; new tool registry and permissions.
Research Agent	Support agent state/tool loop becomes the research state/tool loop.
Ask-the-Web	Search adapter, Source objects, parallel execution, evidence store and citation formatter.
Deep Research	Planning, verification and failure dataset. Failures become evaluation/training data.
Fine-tuning	Same model interface; adapted model becomes another backend.
Serving	vLLM endpoint plugs into the existing model interface.
MCP/A2A	Expose existing tools/agents; do not duplicate business logic.
Capstone	Assembly and hardening, not a new greenfield stack.

Portfolio strategy

Not every project is a flagship. The first projects are learning laboratories; the later projects are evidence of engineering depth.

Project	Approx. size	Portfolio role	Proof
Document Processing Backend	4 weeks	Portfolio	Backend engineering
Spam Classifier	2–3 days	Lab	ML evaluation
Image Classifier	3–4 days	Lab	PyTorch
Tiny Autocomplete + BPE	~1 week	Lab	Language model fundamentals
Mini-GPT	~1 week	Lab	Transformer understanding
LLM Playground	1 week	Portfolio extension	Model abstraction/inference
Berkshire Research Assistant v2	5 weeks	Flagship	RAG + evaluation
Customer Support Agent	2 weeks	Portfolio	Tools + state + permissions
Ask-the-Web / Research Agent	2 weeks	Flagship	Agent engineering
Deep Research	2 weeks	Flagship	Planning + verification
Fine-tuned Model	1 week	Portfolio	SFT/LoRA
Model Serving	1 week	Portfolio	Inference engineering
MCP/A2A labs	2 weeks	Extension	Interoperability
Production AI Capstone	4 weeks	Flagship	End-to-end system

Week 1 — HTTP, FastAPI and the first real API

Main project: Document Processing Backend

Why this week exists: Create the service boundary every later AI application will use.

Concepts to focus on

- HTTP lifecycle
- REST resources
- FastAPI routing/dependency injection
- Pydantic validation
- status codes/error contracts
- OpenAPI

Tools

- Python 3.12+
- FastAPI
- Pydantic
- pytest/httpx

Daily plan

Day	Task / output
Day 1	Write a one-page API spec for users/documents; create repo, venv and /health.
Day 2	Learn HTTP methods/status codes/headers/JSON; implement GET /health and GET /documents with in-memory data.
Day 3	Create Pydantic request/response schemas; reject malformed payloads.
Day 4	Split routers/services; implement POST /documents and GET /documents/{id}.
Day 5	Implement update/delete; add 404/409/422 behavior; inspect OpenAPI.
Day 6	Write 10+ API tests covering happy paths and invalid requests.

Folder Structure

```
Python
doc-processor/
├── src/doc_processor
│   ├── __init__.py
│   ├── main.py
│   └── api/
│       └── __init__.py
```

```

|   |   └─ v1/
|   |       └─ __init__.py
|   |       └─ router.py
|   |       └─ routes/
|   |           └─ __init__.py
|   |           └─ health.py
|   |           └─ auth.py
|   |           └─ documents.py
|   └─ core/
|       └─ __init__.py
|       └─ config.py
|       └─ security.py
|   └─ models/
|       └─ __init__.py
|   └─ schemas/
|       └─ __init__.py
|   └─ services/
|       └─ __init__.py
|   └─ repositories/
|       └─ __init__.py
|   └─ db/
|       └─ __init__.py
└─ tests/
    └─ unit/
    └─ integration/
└─ docs/
    └─ api-spec.md

```

```
|— .env.example
|— .gitignore
|— pyproject.toml
|— README.md
└— docker-compose.yml
```

Completion gate

- You can explain HTTP/REST.
- Every endpoint validates input and returns predictable errors.
- Tests run from a clean environment.

How this connects forward

The API structure becomes the service layer for Berkshire and every later agent.

Week 2 — PostgreSQL, data modeling and authentication

Main project: Document Processing Backend

Why this week exists: Persist the application and make it multi-user.

Concepts to focus on

- relational modeling
- keys/constraints
- indexes
- transactions
- SQLAlchemy
- Alembic
- password hashing
- JWT
- connection management

Tools

- PostgreSQL
- SQLAlchemy
- Alembic
- JWT/auth library

Daily plan

Day	Task / output
Day 1	Design users/documents schema in SQL first; identify constraints/indexes.
Day 2	Connect PostgreSQL; create models and first migration.
Day 3	Replace in-memory CRUD with PostgreSQL; test rollback behavior.
Day 4	Implement registration/login/password hashing and auth dependency.
Day 5	Scope document access to authenticated user; test cross-user access.
Day 6	Add migration/seed workflow and integration tests.

Completion gate

- You can write core SQL without ORM magic.
- Auth is tested including unauthorized access.
- A new developer can initialize DB from migrations.

How this connects forward

PostgreSQL, auth and migrations are reused for documents, chunks, research runs and users.

Week 3 — Files, background processing and async boundaries

Main project: Document Processing Backend

Why this week exists: PDFs need parsing; later RAG ingestion becomes this pipeline.

Concepts to focus on

- multipart upload
- file validation
- async I/O
- background jobs
- job states
- idempotency
- retries

Tools

- FastAPI UploadFile
- PyMuPDF/pypdf
- Celery/RQ/Arq or simple worker abstraction

Daily plan

Day	Task / output
Day 1	Add PDF upload; validate size/type; store with generated ID.
Day 2	Create processing_jobs with queued/processing/completed/failed states.
Day 3	Implement worker: extract text and metadata; persist results.
Day 4	Make retries idempotent; prevent duplicate extraction/corruption.
Day 5	Add job-status endpoint; test slow/failing processing.
Day 6	Run upload → queued → processing → completed → retrieve text end to end.

Completion gate

- Failed jobs are visible and retryable.
- HTTP request does not block on large parsing work.
- You can explain the async boundary.

How this connects forward

The exact pipeline is reused to ingest Berkshire letters.

Week 4 — Docker, tests, logging and performance baseline

Main project: Document Processing Backend

Why this week exists: Make the backend reproducible and measurable.

Concepts to focus on

- Docker
- environment config
- structured logging
- integration tests
- health/readiness
- load testing
- p50/p95 latency

Tools

- Docker
- docker compose
- pytest
- httpx
- Locust or k6

Daily plan

Day	Task / output
Day 1	Containerize API; add .env.example; remove secrets from Git.
Day 2	Compose API + PostgreSQL + worker; verify startup from empty environment.
Day 3	Add request/document/job IDs to structured logs; add /health and /ready.
Day 4	Write integration tests for upload, DB, processing, retry and auth.
Day 5	Load-test read and upload endpoints; record p50/p95 and failure rate.
Day 6	Clean README: architecture, setup, tests, benchmark and limitations.

Completion gate

- Fresh clone → compose up → tests pass.
- README explains architecture/tradeoffs.
- A performance baseline exists.

How this connects forward

This is the reusable backend skeleton; later projects inherit it instead of rebuilding it.

Week 5 — Classical ML: Spam Classifier

Main project: Spam Classifier (short lab)

Why this week exists: Learn the ML lifecycle before neural networks and carry evaluation discipline forward.

Concepts to focus on

- features
- train/validation/test
- baseline
- TF-IDF
- logistic regression
- precision/recall/F1
- confusion matrix
- leakage

Tools

- scikit-learn
- pandas
- numpy
- matplotlib

Daily plan

Day	Task / output
Day 1	Choose a public SMS spam dataset; inspect balance and define the target/metric.
Day 2	Build majority baseline and TF-IDF + logistic regression; compare accuracy/F1.
Day 3	Inspect false positives/negatives; change one preprocessing choice.
Day 4	Create tiny inference script/API; save vectorizer + model.
Day 5	Test leakage, duplicates and class imbalance; fix issues.
Day 6	Write report: dataset, baseline, model, metrics, failures, next experiment.

Completion gate

- You understand why accuracy can mislead.
- You can explain precision/recall from errors.
- Training/evaluation is reproducible.

How this connects forward

The experiment habit becomes the basis for retrieval/agent evaluations.

Week 6 — PyTorch: Image Classifier

Main project: Image Classifier (short lab)

Why this week exists: Build intuition for tensors, neural networks, autograd and optimization.

Concepts to focus on

- tensors
- Dataset/DataLoader
- transforms
- model
- loss
- backprop
- optimizer
- train/validation
- overfitting

Tools

- PyTorch
- torchvision
- Matplotlib

Daily plan

Day	Task / output
Day 1	Use Fashion-MNIST; inspect tensor shapes, labels and batches.
Day 2	Implement a small MLP and training loop; print loss/accuracy.
Day 3	Add validation; plot train vs validation loss and identify overfit.
Day 4	Add one controlled transform/regularization change and compare.
Day 5	Inspect wrong predictions; save/load model; run single-image inference.
Day 6	Write your own forward/loss/gradient/optimizer explanation.

Completion gate

- You can trace tensor shapes.
- You can explain autograd/backprop.
- You can diagnose basic overfitting.

How this connects forward

PyTorch concepts later make fine-tuning and model training less opaque.

Week 7 — Language modeling + Tiny Autocomplete + BPE

Main project: Tiny Autocomplete + tokenizer lab

Why this week exists: Understand token IDs, embeddings and next-token prediction before Transformers.

Concepts to focus on

- vocabulary
- token IDs
- n-grams
- character LM
- probability
- cross-entropy
- embeddings
- BPE idea

Tools

- Python
- PyTorch
- Hugging Face Tokenizers

Daily plan

Day	Task / output
Day 1	Build a character bigram model from a small text corpus.
Day 2	Generate text from transition probabilities; inspect failures.
Day 3	Replace counts with a tiny neural next-character model and train it.
Day 4	Create encode/decode and compare character vs word vocabulary.
Day 5	Train a small BPE tokenizer with Hugging Face Tokenizers; inspect tokens/merges.
Day 6	Write why subwords exist and how token count affects context/cost.

Completion gate

- You can explain next-token prediction.
- You understand tokenization/context length.
- You can inspect token IDs and decode them.

How this connects forward

The tokenizer and next-token concepts feed directly into Mini-GPT and real LLM inference.

Week 8 — Transformer foundations: build Mini-GPT

Main project: Mini-GPT (educational)

Why this week exists: Understand the mechanism underneath the LLM APIs you will use.

Concepts to focus on

- embeddings
- positions
- self-attention
- causal mask
- Q/K/V
- multi-head attention
- MLP block
- residuals
- layer norm
- sampling

Tools

- PyTorch

Daily plan

Day	Task / output
Day 1	Implement token/position embeddings; draw the forward pass and inspect shapes.
Day 2	Implement single-head causal attention and verify future tokens are masked.
Day 3	Add multi-head attention, residuals and feed-forward block; train tiny corpus.
Day 4	Add validation loss/checkpoints; change context length/model size.
Day 5	Implement temperature/top-k/top-p; compare outputs.
Day 6	Write README explaining every component and what Mini-GPT does not teach.

Completion gate

- You can explain Q/K/V and causal masking.
- You can identify embeddings/attention/MLP.
- The model trains and generates from a checkpoint.

How this connects forward

This gives you the mental model needed to use real models intelligently in Month 3.

Week 9 — LLM Playground: real model inference

Main project: LLM Playground

Why this week exists: Move from implementing internals to engineering around real models.

Concepts to focus on

- Hugging Face inference
- model/tokenizer interface
- base vs instruct
- generation parameters
- structured output
- model abstraction

Tools

- Transformers
- a small instruct model suitable for your hardware
- Pydantic

Daily plan

Day	Task / output
Day 1	Load a small open instruct model; inspect tokenization and generation.
Day 2	Compare greedy/temperature/top-k/top-p on fixed prompts.
Day 3	Create provider-neutral generate(messages,params) interface.
Day 4	Add structured Pydantic output; handle malformed responses.
Day 5	Build 20-question mini eval and record latency/quality.
Day 6	Refactor so Berkshire can call an LLM without knowing the provider/model.

Completion gate

- Provider is swappable by configuration.
- You understand generation parameters.
- A repeatable mini-eval exists.

How this connects forward

The model adapter becomes the LLM layer for Berkshire, Support, Research and the capstone.

Week 10 — Berkshire RAG: ingestion + BM25 baseline

Main project: Warren Buffett Research Assistant v2

Why this week exists: Reuse Month 1 document infrastructure and build RAG from primitives.

Concepts to focus on

- corpus design
- PDF cleaning
- chunking
- metadata
- BM25
- top-k
- source IDs

Tools

- Month 1 backend
- PyMuPDF/pypdf
- PostgreSQL
- rank_bm25 or equivalent

Daily plan

Day	Task / output
Day 1	Define Berkshire corpus and canonical document schema: year/source/page.
Day 2	Reuse document processing; add deterministic document IDs and metadata.
Day 3	Implement chunking; compare roughly 256/512/1024-token chunks.
Day 4	Build BM25 retrieval returning chunks + metadata.
Day 5	Create 30–50 questions with expected source/page where possible.
Day 6	Measure Recall@k/MRR and label retrieval failure reasons.

Completion gate

- Evidence is retrievable without an LLM.
- Every chunk traces to document/page/year.
- A fixed evaluation set exists.

How this connects forward

This is the first major reuse of Month 1.

Week 11 — Berkshire RAG: embeddings, hybrid search, reranking

Main project: Warren Buffett Research Assistant v2

Why this week exists: Learn actual retrieval engineering instead of treating vector search as a checkbox.

Concepts to focus on

- embeddings
- cosine similarity
- dense retrieval
- BM25
- hybrid/RRF
- reranking
- filters

Tools

- Sentence Transformers
- Qdrant or pgvector
- BM25
- optional reranker

Daily plan

Day	Task / output
Day 1	Generate dense embeddings and store vectors + metadata.
Day 2	Implement dense retrieval and compare against BM25.
Day 3	Implement hybrid retrieval with RRF or weighted fusion.
Day 4	Add reranking over top candidates; keep retrieval and reranking separate.
Day 5	Run controlled experiments: chunk size, k, BM25, dense, hybrid, rerank.
Day 6	Write decision report with quality/latency/storage tradeoffs.

Completion gate

- You understand why keyword and dense search fail differently.
- You measured retrieval.
- Retriever has a reusable interface.

How this connects forward

Retriever becomes a shared knowledge tool for Support and Research.

Week 12 — Berkshire RAG: grounded generation + citations

Main project: Warren Buffett Research Assistant v2

Why this week exists: Connect retrieval to the LLM while controlling evidence.

Concepts to focus on

- context assembly
- prompt contracts
- grounding
- citation mapping
- abstention
- context limits

Tools

- Month 9 LLM adapter
- Pydantic
- retriever

Daily plan

Day	Task / output
Day 1	Design response schema: answer, citations, grounding status, sources.
Day 2	Build context assembly with source IDs/page/year.
Day 3	Create grounded prompt: answer only from evidence; abstain when insufficient.
Day 4	Implement citation mapping from claims/answer to sources.
Day 5	Test unanswerable, conflicting, irrelevant and malicious-document cases.
Day 6	Compare no-RAG vs BM25-RAG vs hybrid-RAG on fixed eval set.

Completion gate

- Answers trace to evidence.
- Unsupported questions can be refused.
- RAG beats a naive baseline on measured tasks.

How this connects forward

This turns your existing Berkshire project into a serious RAG system.

Week 13 — Berkshire RAG: evaluation + productionization

Main project: Warren Buffett Research Assistant v2

Why this week exists: Make RAG measurable, traceable and reusable.

Concepts to focus on

- offline evals
- retrieval metrics
- answer correctness
- faithfulness/grounding
- citation coverage
- tracing
- latency
- cost

Tools

- Ragas or custom evaluators
- Langfuse
- pytest
- Docker

Daily plan

Day	Task / output
Day 1	Expand to 50–100 questions: single-hop, multi-hop, comparison, temporal, unanswerable.
Day 2	Implement retrieval + answer/citation evaluators; separate retrieval from generation failures.
Day 3	Trace retrieval, prompts, model calls and final output; record latency/tokens.
Day 4	Build experiment runner: dataset → variant → metrics → report.
Day 5	Add caching, timeouts, rate limiting and p50/p95 measurements.
Day 6	Freeze v2.0 README with architecture, experiments, metrics and failure analysis.

Completion gate

- Changes require experiments.
- You can localize bad answers to retrieval or generation.
- Repository is interview-ready.

How this connects forward

This becomes the knowledge/evaluation subsystem for later agents.

Week 14 — Customer Support Agent: RAG + tools

Main project: Customer Support Agent

Why this week exists: Berkshire answers from knowledge; Support adds safe actions and permissions.

Concepts to focus on

- tool/function calling
- structured schemas
- deterministic tools
- authorization
- tool results
- agent vs workflow

Tools

- FastAPI
- Pydantic
- Berkshire retriever
- tool-calling LLM
- LangGraph later

Daily plan

Day	Task / output
Day 1	Design fake support domain: users/orders/refunds/tickets.
Day 2	Implement deterministic tools: get_order, refund_status, create_ticket, search_policy.
Day 3	Write one hand-coded tool-calling loop before using a framework.
Day 4	Add auth-derived user identity; model cannot choose identity.
Day 5	Connect Berkshire-style RAG as search_policy/knowledge tool.
Day 6	Test wrong args, unauthorized IDs, repeated calls and tool failures.

Completion gate

- LLM cannot bypass authorization.
- Tools are independently testable.
- You can explain code-vs-LLM decisions.

How this connects forward

Berkshire retriever is reused instead of rebuilt.

Week 15 — Customer Support Agent: state, memory, retries, human handoff

Main project: Customer Support Agent

Why this week exists: Learn stateful orchestration that will become the research-agent foundation.

Concepts to focus on

- state
- agent loop
- persistence
- memory
- retries
- timeouts
- human-in-loop
- streaming

Tools

- LangGraph
- persistence/checkpointing
- Langfuse

Daily plan

Day	Task / output
Day 1	Design state machine before framework: intake → classify → retrieve/tool → respond → escalate/end.
Day 2	Implement graph and state.
Day 3	Add checkpointing and resume.
Day 4	Add max tool calls, retry/timeout policies and failure taxonomy.
Day 5	Add human escalation and streaming progress.
Day 6	Trace 20 conversations; convert failures into regression tests.

Completion gate

- Every state transition is inspectable.
- No infinite tool loop.
- Escalation is auditable.

How this connects forward

State/tool loop is generalized into research.

Week 16 — Tool-Using Research Agent

Main project: Tool-Using Research Agent

Why this week exists: Generalize support tools into research capabilities.

Concepts to focus on

- tool selection
- state
- planning vs acting
- ReAct loop
- observation
- tool budgets
- evidence objects

Tools

- LangGraph
- search/mock search
- Berkshire retriever
- Langfuse

Daily plan

Day	Task / output
Day 1	Define search_documents, open_source, calculate tools with strict schemas.
Day 2	Build bounded tool loop and log actions/observations.
Day 3	Create reusable Source/Evidence objects.
Day 4	Add planning for multi-step questions.
Day 5	Add tool budget, timeouts and retries; test loops.
Day 6	Run 20 tasks; measure success, tool count, latency and failures.

Completion gate

- State is explicit.
- Tool calls are bounded/observable.
- Research outputs include evidence.

How this connects forward

The same state/tool abstraction now supports web research.

Week 17 — Ask-the-Web: decomposition + parallel search

Main project: Ask-the-Web Agent

Why this week exists: Move from a controlled corpus to web sources and learn parallel orchestration.

Concepts to focus on

- search adapter
- query decomposition
- parallelization
- routing
- source quality
- deduplication
- synthesis

Tools

- Search API
- LangGraph
- FastAPI
- Pydantic

Daily plan

Day	Task / output
Day 1	Build search adapter returning normalized Source objects.
Day 2	Implement single search + citation formatting + URL deduplication.
Day 3	Add query decomposition; compare one query vs targeted queries.
Day 4	Run independent searches in parallel and measure latency.
Day 5	Add source-quality filtering and evidence extraction.
Day 6	Evaluate 20 questions for correctness, coverage, latency, tool calls.

Completion gate

- Sources support claims.
- Parallel search reduces tested latency.
- Partial search failures do not crash the run.

How this connects forward

Source/evidence/citation code is reused by Deep Research.

Week 18 — Deep Research: planning + execution

Main project: Deep Research Agent

Why this week exists: Teach the system to decide what it needs to know and track long-running research.

Concepts to focus on

- research plans
- subtasks
- dependencies
- budgeting
- context management
- iterative search

Tools

- LangGraph
- search adapter
- Berkshire retriever
- PostgreSQL

Daily plan

Day	Task / output
Day 1	Define ResearchTask schema: question, subquestions, evidence requirements, status, budget.
Day 2	Implement structured planner.
Day 3	Execute independent subtasks in parallel.
Day 4	Add dependent subtasks that consume earlier findings.
Day 5	Implement context compaction: preserve evidence/conclusions, discard redundancy.
Day 6	Run 10 complex questions and inspect missed evidence.

Completion gate

- Plans are executable.
- Failed subtasks can be retried/continued.
- Context does not grow without bound.

How this connects forward

Research failures become the data source for evaluation and fine-tuning.

Week 19 — Deep Research: verification + evidence grading

Main project: Deep Research Agent

Why this week exists: Make research quality measurable rather than judged by prose.

Concepts to focus on

- claim/evidence mapping
- verification
- self-refinement
- source diversity
- offline evals
- LLM judge limitations

Tools

- Langfuse
- Ragas/custom evaluators
- pytest

Daily plan

Day	Task / output
Day 1	Represent final answer as claims + evidence references.
Day 2	Build verifier checking claim support.
Day 3	Add source diversity/quality checks.
Day 4	Create evaluation set with expected claims/evidence where practical.
Day 5	Compare single-pass vs planner+parallel vs planner+verification.
Day 6	Select 10 highest-value failures for training/eval data.

Completion gate

- Unsupported claims are detected/flagged.
- Architecture variants are compared with metrics.
- Dataset is versioned and reusable.

How this connects forward

Evaluation data now has a purpose beyond testing: it can improve the model/system.

Week 20 — Fine-tuning: SFT + LoRA

Main project: Fine-tuned Small LLM

Why this week exists: Learn when adaptation is justified using failures from your own systems.

Concepts to focus on

- instruction data
- train/validation
- SFT
- LoRA/PEFT
- QLoRA concept
- overfitting
- baseline comparison

Tools

- Transformers
- Datasets
- PEFT
- TRL SFTTrainer
- small open instruct model

Daily plan

Day	Task / output
Day 1	Choose one narrow behavior: citation formatting, research-plan generation or support classification.
Day 2	Create 300–1000 high-quality examples from failures; split train/validation.
Day 3	Run base-model baseline and define exact metrics.
Day 4	Train LoRA/SFT and record configuration/compute/time.
Day 5	Evaluate held-out/adversarial cases; check regressions and memorization.
Day 6	Package adapter + inference script + model card explaining limitations.

Completion gate

- You can justify fine-tuning vs prompting/RAG.
- Held-out evaluation exists.
- You report limitations, not only training loss.

How this connects forward

The adapted model plugs into the same model interface.

Week 21 — Serve the model: vLLM + quantization + benchmarks

Main project: Model Serving Service

Why this week exists: Make the model a backend component and learn inference operations.

Concepts to focus on

- OpenAI-compatible serving
- streaming
- concurrency
- batching
- quantization
- latency
- throughput
- VRAM

Tools

- vLLM
- Docker
- OpenAI-compatible client
- benchmark script

Daily plan

Day	Task / output
Day 1	Serve a compatible small model with vLLM and call it with an OpenAI-style client.
Day 2	Point Month 9 model interface at local endpoint; no app rewrite.
Day 3	Measure single-request latency and tokens/sec; add streaming.
Day 4	Benchmark concurrent requests; record p50/p95/throughput.
Day 5	Test quantized model if hardware supports it; compare memory/quality/speed.
Day 6	Document benchmark method, limits and security assumptions.

Completion gate

- Provider/model is swappable.
- You understand latency vs throughput.
- Benchmarks are reproducible.

How this connects forward

The local model becomes a drop-in inference backend for the final capstone.

Week 22 — MCP: expose existing capabilities

Main project: Berkshire/Research MCP Server

Why this week exists: Learn interoperability by exposing real tools you already own.

Concepts to focus on

- host/client/server
- tools
- resources
- schemas
- capability negotiation
- security/consent

Tools

- MCP SDK
- existing Berkshire/research tools

Daily plan

Day	Task / output
Day 1	Read MCP architecture and map host/client/server onto your system.
Day 2	Expose search_documents as an MCP tool.
Day 3	Expose source/document resources.
Day 4	Add permissions/input validation and consider prompt injection.
Day 5	Connect a compatible MCP client/host and run real tasks.
Day 6	Write protocol-vs-application comparison and security notes.

Completion gate

- You can explain MCP precisely.
- Tools have schemas and permissions.
- Direct and MCP calls reuse the same implementation.

How this connects forward
MCP is an interface around existing capabilities, not a separate toy app.

Week 23 — A2A + remote agent boundaries

Main project: Research Agent as a Remote Agent

Why this week exists: Learn when an agent should be a service boundary.

Concepts to focus on

- Agent Card
- Tasks
- Messages
- Parts/Artifacts
- remote agent
- capability discovery
- HTTP/JSON-RPC
- communication overhead

Tools

- A2A reference/SDK
- FastAPI
- existing research agent

Daily plan

Day	Task / output
Day 1	Read A2A concepts; decide what should remain internal vs remote.
Day 2	Create capability/agent description for one narrow skill.
Day 3	Implement client request + task lifecycle + structured artifact.
Day 4	Measure remote overhead vs local function.
Day 5	Test remote timeout/unavailable agent/invalid task/partial result.
Day 6	Decide whether A2A belongs in the final architecture and document why.

Completion gate

- You understand remote boundaries.
- You can distinguish A2A from MCP.
- You can justify not using multi-agent complexity.

How this connects forward

The lesson is service design, not collecting protocols.

Week 24 — AI observability, evaluation and CI

Main project: AI Engineering Evaluation Layer

Why this week exists: Turn evaluation into a reusable subsystem.

Concepts to focus on

- traces
- spans
- datasets
- experiments
- evaluators
- regression thresholds
- prompt/model versioning
- cost/latency

Tools

- Langfuse
- OpenTelemetry concepts
- pytest
- GitHub Actions

Daily plan

Day	Task / output
Day 1	Define trace schema for model/retrieval/tools/output/latency/tokens.
Day 2	Instrument Berkshire and identify top failure modes.
Day 3	Create reusable datasets for Berkshire/Support/Research.
Day 4	Create experiments comparing model/prompt/retriever variants.
Day 5	Add CI regression checks with thresholds.
Day 6	Build quality report/dashboard for correctness, grounding, citation, latency, cost.

Completion gate

- Evals can run without manual UI work.
- CI can block a regression.
- Traces identify where bad answers originate.

How this connects forward

This becomes a shared quality layer for the capstone.

Week 25 — Production AI capstone: architecture + vertical slice

Main project: Production AI Research Platform

Why this week exists: Assemble the best pieces from six months.

Concepts to focus on

- service boundaries
- auth
- queues
- RAG
- agents
- model abstraction
- caching
- rate limits
- security

Tools

- FastAPI
- PostgreSQL
- Redis
- Docker
- LangGraph
- Qdrant/pgvector
- Langfuse
- hosted + local model

Daily plan

Day	Task / output
Day 1	Choose use case; write requirements, non-goals, threat model and architecture.
Day 2	Create repo/shared modules; integrate Month 1 API/DB/auth patterns.
Day 3	Integrate Berkshire retrieval/evidence objects.
Day 4	Integrate agent/tool orchestration; keep deterministic actions in code.
Day 5	Integrate model abstraction + optional local endpoint; add caching/timeouts.
Day 6	Integrate traces/evals and run one complete user journey.

Completion gate

- One vertical slice works end-to-end.
- Components have clear contracts.
- Failure behavior is explicit.

How this connects forward

This is integration, not another greenfield tutorial.

Week 26 — Hardening, deployment, portfolio and interviews

Main project: Production AI Research Platform

Why this week exists: Turn six months of learning into evidence an employer can inspect.

Concepts to focus on

- security
- secrets
- authorization
- rate limiting
- reliability
- deployment
- CI/CD
- documentation
- system design
- interview storytelling

Tools

- Docker
- GitHub Actions
- chosen cloud/local deployment
- Langfuse
- pytest

Daily plan

Day	Task / output
Day 1	Security pass: secrets, auth, file limits, prompt injection boundaries, SSRF/tool risks, sensitive logs.
Day 2	Reliability pass: timeouts, retries, queue failure, DB constraints, idempotency.
Day 3	Performance pass: cache, reduce unnecessary LLM calls, benchmark concurrency.
Day 4	Deploy; add CI for lint/tests/evals/build; verify clean deployment.
Day 5	Portfolio pass: pin repos; rewrite READMEs around problem/architecture/experiments/results.
Day 6	Interview pass: prepare 20 questions per flagship; practice one tradeoff, failure, metric and debugging story.

Completion gate

- A stranger can understand/run it from README.
- You can defend architecture and metrics.
- GitHub demonstrates backend + AI engineering depth.

How this connects forward

The final output is a coherent portfolio, not a collection of course certificates.

Flagship project playbooks

A. Berkshire Research Assistant v2

Goal: answer questions about Berkshire shareholder letters/annual reports with evidence-backed responses.

Reuse: Month 1 upload/PDF processing, PostgreSQL, jobs, auth, Docker/tests; Month 9 model adapter.

Build sequence: corpus → cleaning → metadata → chunking → BM25 → dense → hybrid → reranking → context assembly → grounded generation → citations → abstention → eval → tracing → optimization.

Evaluation set: 50–100 questions including factual lookup, multi-document comparison, temporal change, multi-hop and unanswerable questions.

Metrics: Recall@5/10, MRR, answer correctness, citation coverage/precision, faithfulness/grounding, p50/p95 latency, token/cost per request.

Failure tests: answer absent, wrong year, conflicting evidence, irrelevant high-score chunk, malicious instruction in source, long multi-document question.

B. Customer Support Agent

Goal: answer policy questions and execute safe support actions.

Connection: Berkshire is the knowledge layer; Support adds transactional tools.

Architecture: User → FastAPI/Auth → Agent State → {Knowledge Search, Order DB, Refund Status, Ticket Tool} → Response/Escalation.

Rules: identity comes from auth; tools validate arguments; destructive actions require deterministic authorization; tools have timeouts/retry limits; sensitive cases escalate.

Evaluate: task success, tool-selection accuracy, invalid-tool-call rate, unauthorized-action rate, escalation accuracy and latency.

C. Ask-the-Web + Deep Research

Ask-the-Web: question → decomposition → parallel search → source normalization → evidence extraction → synthesis.

Deep Research: question → plan → parallel subtasks → dependent subtasks → evidence store → verification → synthesis.

Design rule: add an agent/service only when there is a real capability/state/ownership boundary.

Evaluate: answer correctness, evidence/citation coverage, source diversity, tool count, latency, failures and cost.

D. Fine-tuned model + serving

Start from a narrow behavior produced by failures in your systems.

Baseline: prompted base model. Adaptation: SFT + LoRA. Evaluation: held-out + adversarial + regression set. Serving: vLLM or appropriate server.

Benchmark: latency, tokens/sec, concurrency, memory/VRAM and quality.

The point is to test whether adaptation is useful, not to claim that fine-tuning makes a model universally smarter.

E. Production AI Research Platform

Suggested capabilities: auth, document ingestion, private-document RAG, web research, tools, evidence/citations, background jobs, model-provider abstraction, optional local model, eval dataset, tracing, caching, rate limiting and audit logs.

Non-goals: unnecessary microservices, multi-agent diagrams without need, elaborate frontend, unmeasured claims of scale.

Required artifacts: architecture diagram, API contract, DB schema, threat model, eval dataset, benchmark report, failure analysis and deployment instructions.

Month 2: why these ML projects are included

The screenshot projects are kept where they create a useful mental dependency. They are deliberately short. They are not intended to become polished portfolio products.

Project	What it teaches	Why you need it later
Spam Classifier	dataset splits, baseline, metrics, leakage	same experiment discipline later used for RAG/agents
Image Classifier	tensors, DataLoader, model, loss, backprop, optimizer	understand PyTorch and fine-tuning instead of treating training as magic
Tiny Autocomplete	probability, vocabulary, embeddings, next-token prediction	understand what a language model is predicting
BPE Tokenizer	subword tokenization and token IDs	understand LLM inputs/context/cost
Mini-GPT	causal attention, Q/K/V, decoder block, sampling	understand Transformer internals before using open LLMs

Technology relevance guide

Durable concepts come first. Tool choices are intentionally limited so you learn depth instead of collecting frameworks.

Technology	Priority	Decision
FastAPI	Core	Use throughout the course.
PostgreSQL	Core	Use as primary relational DB.
Docker	Core	Use from Month 1 onward.
PyTorch	Core	Use for ML/Transformer/fine-tuning understanding.
Hugging Face Transformers	Core	Use for real model inference.
Sentence Transformers	Core	Use for embeddings/retrieval.
Qdrant or pgvector	Core	Choose one; understand retrieval interface.
LangGraph	Useful	Use after hand-coding an agent loop; current docs focus on durable execution, persistence, streaming and human-in-loop.
Langfuse	Useful	Use for traces/datasets/experiments/evals.
PEFT/LoRA	Core for AI engineer	Use for practical model adaptation.
TRL	Useful	Use for SFT after understanding the baseline.
vLLM	Core exposure	Use for model serving/benchmarking.
MCP	Relevant extension	Learn by exposing existing tools.
A2A	Relevant extension	Learn as remote-agent/service-boundary concept; do not force it into the capstone.
Diffusion/image generation	Optional	Keep as breadth; lower priority than RAG, agents, evaluation and inference for your target roles.

Parallel track: use AI to work faster without losing engineering skill

Weeks	Practice
1–4	Ask AI to explain errors, generate tests after you define behavior, review SQL and propose edge cases. Never merge without tests.
5–8	Use AI to explain tensor shapes, debug training loops, compare experiments and generate fixtures. Re-derive important concepts yourself.
9–13	Use AI to create candidate eval questions, classify retrieval failures, inspect traces and propose experiments. You remain the evaluator.
14–19	Use AI to inspect agent traces, find loops, generate adversarial tool inputs and suggest simpler state machines.
20–22	Use AI for dataset cleaning, training configuration review, benchmark analysis and deployment troubleshooting.
23–26	Use AI for architecture review, threat models, API contracts, CI failures and documentation. Practice repository-level tasks with acceptance criteria.

AI coding-agent protocol

1. Write the task, inputs, outputs, constraints and acceptance tests before asking AI to code.
2. Ask it to inspect relevant files before modifying anything.
3. Make small changes rather than one giant generated rewrite.
4. Run tests and inspect the diff.
5. Ask for an explanation of code you cannot explain.
6. Keep architecture and design decisions human-owned.

Parallel interview track: DSA + system design

- Weeks 1–4: arrays, strings, hash maps, two pointers, sliding window, stacks/queues.
- Weeks 5–8: binary search, linked lists, recursion, trees, heaps and basic graphs.
- Weeks 9–13: BFS/DFS, intervals, greedy, prefix sums and backtracking.
- Weeks 14–19: graphs, topological sort, union-find and dynamic programming basics.
- Weeks 20–26: mixed interview sets + backend/system design: file processing service, rate limiter, job queue, RAG service and AI research system.
- For each flagship project prepare a 5-minute architecture explanation and a 10-minute deep dive.

Six-month completion checklist

- ☐ Build a REST API with FastAPI and predictable error contracts.
- ☐ Model relational data in PostgreSQL and use migrations.
- ☐ Implement authentication and authorization.
- ☐ Process files asynchronously with retries/idempotency.
- ☐ Write unit/integration tests and containerize the service.
- ☐ Build/evaluate a classical ML classifier with an appropriate metric.
- ☐ Use PyTorch tensors, datasets, models, autograd and optimization.
- ☐ Explain tokenization, embeddings, next-token prediction and causal self-attention.
- ☐ Implement/train a tiny decoder-only Transformer.
- ☐ Use a real open/hosted model behind a provider-neutral interface.
- ☐ Implement BM25 and dense retrieval; understand hybrid/reranking.
- ☐ Ground answers and map citations to evidence.
- ☐ Maintain an evaluation dataset and run repeatable experiments.
- ☐ Implement tool calling with strict schemas and backend authorization.
- ☐ Manage agent state, retries, budgets and human escalation.
- ☐ Build web research with decomposition, parallel search and citations.
- ☐ Build Deep Research with planning and verification.
- ☐ Fine-tune a small model with SFT/LoRA and compare against baseline.
- ☐ Serve a model and benchmark latency/throughput.
- ☐ Expose existing tools through MCP.
- ☐ Understand A2A remote-agent boundaries and implement a minimal exchange.
- ☐ Trace LLM calls, retrieval and tools.
- ☐ Run offline evaluations and CI regression checks.
- ☐ Use Docker, secrets, rate limits, caching, timeouts and logging.
- ☐ Maintain five or more strong repositories with clear READMEs.
- ☐ Be able to defend every major architecture decision and metric in an interview.

Flagship interview questions

- Why did you choose BM25, dense, hybrid or reranking?
- How did you measure retrieval improvement?
- What happens when the answer is not in the corpus?
- How do you prevent retrieved text from becoming instructions?
- Why should this decision be deterministic rather than made by the LLM?
- How do you prevent an agent from looping?
- How do you authorize a tool call?
- How would you reduce research-agent latency?
- What would you cache?
- How would you handle 100 concurrent research requests?
- How do you resume a long-running research task?
- Why fine-tune instead of prompting/RAG?
- How did you create the fine-tuning dataset?
- How do you detect overfitting?
- What changes under quantization?
- Latency vs throughput: what is the difference?
- What happens if the embedding model changes?
- How do you version evaluation datasets?
- What belongs in traces and what should be redacted?
- When would MCP help?
- When would A2A be unnecessary?

- How would you deploy and monitor the system?

Official resource index

[PyTorch Learn the Basics](#) — Tensors, datasets, models, autograd, optimization and saving/loading models.

[Hugging Face Transformers](#) — Model loading and inference ecosystem.

[Hugging Face PEFT](#) — Parameter-efficient fine-tuning/LoRA.

[Hugging Face TRL](#) — SFT and post-training tooling.

[FastAPI](#) — API, async and testing.

[LangGraph](#) — Workflows/agents, persistence and streaming.

[Qdrant](#) — Vector, hybrid and multi-stage retrieval.

[Langfuse](#) — Tracing, datasets, experiments and evaluations.

[vLLM](#) — Model serving and OpenAI-compatible APIs.

[MCP](#) — Protocol architecture, tools/resources and capability negotiation.

[A2A](#) — Agent Cards, Tasks, Messages, Parts and remote agent interactions.

Repository standard for every serious project

- README: problem, users, requirements, architecture, setup, API/data, experiments, metrics, limitations and future work.
- Architecture diagram showing API, database, queue, model, retrieval and external services.
- Tests for happy paths, invalid inputs and major failure modes.
- Fixed evaluation dataset wherever output quality matters.
- Decision log for major alternatives and tradeoffs.
- Benchmark for latency/throughput/cost/quality when relevant.
- Security notes covering auth, secrets, file validation, prompt injection/tool risks and sensitive logs.
- Demo command or short video/GIF for the core workflow.
- Pinned dependencies and reproducible setup.

Final principle

Do not optimize GitHub for the number of AI projects. Optimize it for visible progression: backend → ML → LLM internals → RAG → agents → research → fine-tuning → inference → protocols → production.